

Aula Passada

Introdução a Pesquisa Digital

- Árvore Trie
- Árvore Patrícia

Compressão de Dados

Diego Silva Caldeira Rocha

Sumário

Introdução

Teoria da informação

Métricas de Compressão

Algoritmo RLE

Algoritmo Huffman

Algoritmo LZW

Introdução

- Objetivo
 - Codificar um conjunto de informações de modo que o código resultante seja menor que o original
- Justificativa
 - Redução do espaço ocupado
 - Aumento da velocidade de transmissão dos dados
 - Armazenar arquivos grandes (backup): compactação permite armazenamento ocupando mesmo espaço

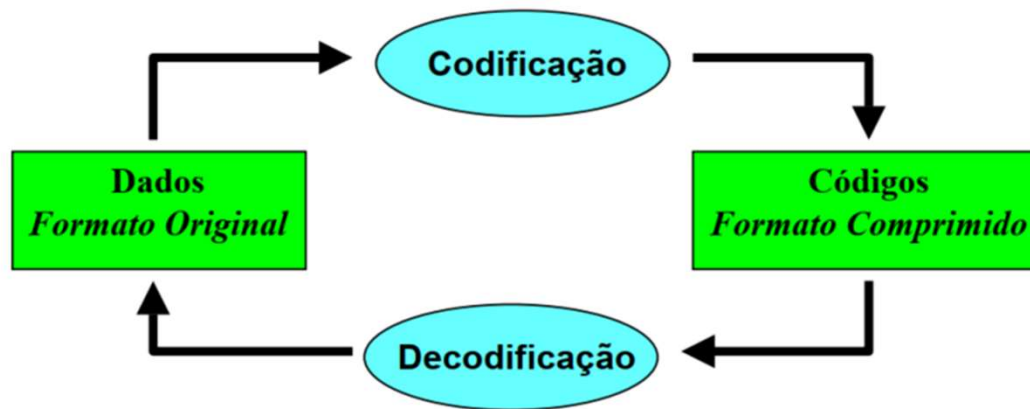
Introdução

- Antes da Compressão
 - Racionalização da representação
 - Eliminação de itens redundantes
 - Aluno {Nome, Matrícula, nota1, nota2, nota3, média}
 - Aluno {Nome, Matrícula, nota1, nota2, nota3}
 - Uso de notação codificada
 - 17 de Abril de 2024 $\Rightarrow$ 19 bytes
 - 17/Abril/2024 $\Rightarrow$ 13 bytes
 - 17/04/2024 $\Rightarrow$ 10 bytes
 - 17/04/24 $\Rightarrow$ 8 bytes
 - 17/04/24 $\Rightarrow$ 3 bytes (Notação Binária)
 - 00010001 00000100 00011000
 - Bits realmente utilizados : 10001 0100 0011000 $\Rightarrow$ 2 byte

Introdução

- Antes da Compressão (continuação)
 - Racionalização da representação
 - Codificação de textos
 - ⇒ Diminuição da ocorrência de textos em tabelas
 - Funcionário {Nome, CódF, NomeDept, DataNasc}
 - Equipamento {Nome, CódE, Desc, NomeDept}
 - Funcionário {Nome, CódF, CodDept, DataNasc}
 - Equipamento {Nome, CódE, Desc, CodDept}
 - Departamento {CodDept, NomeDept}
 - Supressão de espaços inúteis
 - ⇒ Campos com tam. fixo devem ser avaliados com muito cuidado.
 - Ex. Nome: 70 caracteres (José da Silva ⇒ 13 bytes)

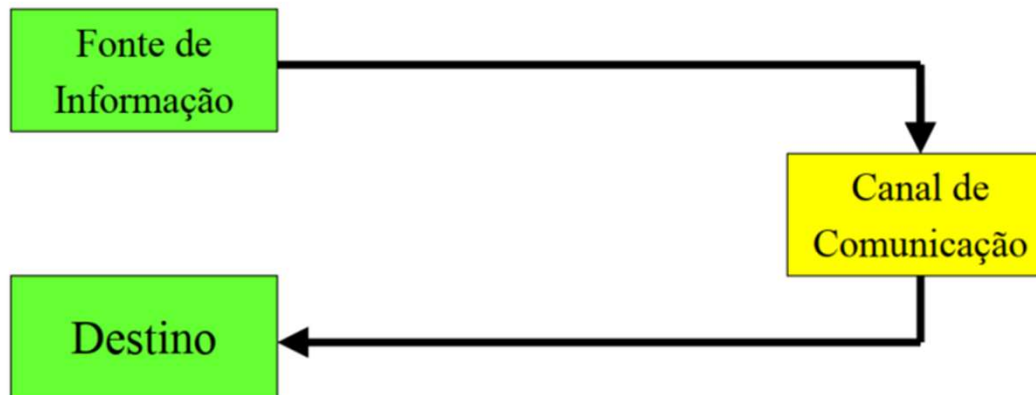
Introdução



Teoria da Informação

- C. Shannon

A mathematical theory of communications. Bell System Tech. Journal, 27:379-423, 1948.



Teoria da Informação

- Exemplo dado de oito faces desonesto

- Seriam necessário 3 bits
- Probabilidades:

$$\begin{array}{llll}
 p_A = 1/2 & p_B = 1/4 & p_C = 1/8 & p_D = 1/16 \\
 p_E = 1/32 & p_F = 1/64 & p_G = 1/128 & p_H = 1/128
 \end{array}$$

- Representação

A - 0	B - 10	C - 110	D - 1110
E - 11110	F - 111110	G - 1111110	H - 1111111

$$\begin{aligned}
 & \frac{1}{2} \times 1 + \frac{1}{4} \times 2 + \frac{1}{8} \times 3 + \frac{1}{16} \times 4 + \\
 & \frac{1}{32} \times 5 + \frac{1}{64} \times 6 + \frac{1}{128} \times 7 + \frac{1}{128} \times 7 = 1,98 \text{ bits}
 \end{aligned}$$



Teoria da Informação

- A entropia mede a quantidade média de informação contida em uma fonte de dados.
- Quanto mais imprevisível é um símbolo, maior é sua entropia.

$$H = - \sum_x p(x) \log_2 p(x)$$

$H(x)$ = Entropia do evento x

$P(x)$ = probabilidade de um evento x acontecer 0 a 1

Como calcular $\log_2$ sem calculadora científica

- Muitas calculadoras não possuem $\log_2$, mas é possível usar a mudança de base:
 - $\log_2(p) = \log_{10}(p) / \log_{10}(2)$ ou $\log_2(p) = \ln(p) / \ln(2)$
- Exemplo:
 - $\log_2(0,045) = \log(0,045) / \log(2) = (-1,3468) / 0,3010 = -4,47$
- Dica: pode-se usar log comum (log) ou log natural (ln) na calculadora.

Teoria da Informação

Caracter	# de ocorrências	Valor ASCII	Valor Binário
T	1	79	0100 1111
R	4	114	0111 0010
O	5	111	0110 1111
A	4	97	0110 0001
(espaço)	8	32	0010 0000

Teoria da Informação

Teoria da Informação — Exemplo de Cálculo de Entropia

Caractere	Ocorrências	Probabilidade (p)
T	1	0,045
R	4	0,182
O	5	0,227
A	4	0,182
(espaço)	8	0,364

 **Total:** 22 caracteres 22 bytes
 **Tamanho original:** $22 \times 8 = 176$ bits

Teoria da Informação

$$H = - \sum p(x) \log_2 p(x)$$

Caractere	p(x)	-p·log ₂ (p)
T	0,045	0,201
R	0,182	0,446
O	0,227	0,485
A	0,182	0,446
(espaço)	0,364	0,530

$$H = 2,108 \text{ bits/símbolo}$$

- Entropia total: $H = 2,108 \text{ bits/símbolo}$
- Mensagem = 22 símbolos
- Bits mínimos necessários: $22 \times 2,108 = 46,4 \text{ bits} \approx 47 \text{ bits}$
- Tamanho original: $22 \times 8 = 176 \text{ bits}$
- Compressão: $176 \text{ bits} \rightarrow 47 \text{ bits}$ (redução $\approx 73\%$)
- Resultado final: $46,4 \text{ bits} \div 8 = 5,8 \text{ bytes} \approx 6 \text{ bytes}$

Métricas para Avaliação de Compressão

- Taxa de Compressão (T_c)

$$T_c = \text{Tamanho Final} \div \text{Tamanho Inicial}$$

- Fator de Compressão (F_c)

$$F_c = \text{Tamanho Inicial} \div \text{Tamanho Final}$$

- Ganho de Compressão (G_c)

$$G_c = 100 \times \log_e (\text{Tamanho de referência} \div \text{Tamanho Final})$$

- Percentual de Redução (P_R)

$$P_R = 100 \times (1 - T_c)$$

Compressão = Modelo + Codificação

- Métodos Estatísticos : Shannon-Fano / Huffman / ...
- Métodos de Dicionário : LZ77 / LZ78 / LZW / ...
- Métodos Preditivos : PPM / JPEG-LS / RLE / ...
- Métodos de Quantização e Transformação : JPEG / MPEG / ...

Compressão de Dados: com perdas

- Detalhes da informação são perdidos durante a descompactação
- Geralmente utilizada na compactação arquivos de multimídia como imagens, áudios e vídeos uma vez que as perdas reduzem a qualidade de forma aceitável, sem afetar a compreensão da informação
- Podem também ser citadas as transmissões ao vivo de áudio e vídeo em que as perdas de qualidade são aceitáveis em favor de uma maior eficiência e desempenho na transmissão dos dados
- Alguns exemplos de algoritmos que se baseiam na compressão com perdas são o JPEG, MP3 e MP4

Compressão de Dados: sem perdas

- Os dados são mantidos íntegros mesmo após os processos de compressão/descompressão
- A compressão sem perdas, é normalmente utilizada na compactação de documentos e outros tipos de informações que precisam permanecer inalteradas após a descompactação
- Duas estratégias mais utilizadas para compressão sem perdas são a codificação de **redundância mínima** e o **método do dicionário**
- **A redundância mínima** consiste em representar os símbolos que aparecem com maior frequência utilizando a menor quantidade de bits para sua representação. Um exemplo de utilização desta estratégia é o algoritmo: **Codificação de Huffman**
- **O método do dicionário** utiliza um arquivo como se fosse um dicionário de expressões com os seus valores correspondentes em bits. Um exemplo de utilização desta estratégia é o algoritmo: **Lempel-Ziv-Welch (LZW)**

Run-Length Encoding (RLE) – supressão de Repetição

Substitui sequências de caracteres repetidos pelo número de ocorrências seguido pelo caractere. Exemplo:

AAAAAHHHFGGGGBBPEEECCCCCDLLLLRR

5A3H1F4G2B1P3E6C1D4L2R (redução de 32 para 22 bytes)

É possível melhorar a compactação definindo que a ausência do número implica em frequência igual a um, resultando no seguinte:

5A3HF4G2BP3E6CD4L2R (redução de 32 para 13 bytes)

Run-Length Encoding (RLE) – supressão de Repetição

No caso de sequências contendo números como por exemplo:

KKKK4444PP888TJJJJ2222NN

É possível adotar algum símbolo especial para resolver esse problema como por exemplo empregar o símbolo @ para indicar que, na sequência, será apresentado um símbolo e não uma frequência, resultando em:

4K4@42P3@8T5J5@22N (redução de 26 para 18 bytes)

Algoritmo de Huffman

- Desenvolvido em 1952 por David A. Huffman e publicado no Artigo: “A Method for the Construction of Minimum-Redundancy Codes”
- Uma das melhores técnicas de compressão conhecidas
- Para uma dada distribuição de probabilidades gera um código ótimo, de redundância mínima, livre de prefixo e produz uma sequência de bits aleatórios
- Utiliza códigos de tamanho variável para representar os símbolos do texto, que podem ser caracteres ou cadeias de caracteres
- A ideia básica do algoritmo é atribuir códigos de bits menores para os símbolos mais frequentes no texto (redundância mínima), e códigos mais longos para os mais raros
- O algoritmo de Huffman original baseia-se no método guloso e constrói um código ótimo com esforço computacional $O(n \cdot \log n)$

Passos para Construir Árvore Trie do Algoritmo de Huffman

- Criar um nó da árvore para cada símbolo de S. Cada nó será considerado uma subárvore.
- Unir as duas subárvores com menor frequência, formando uma nova subárvore
 - A raiz da nova subárvore conterá a soma das frequências das duas subárvores unidas
- Continuar unindo subárvores até que reste apenas uma
- Rotular arestas com ZEROS e UNS (zeros à esquerda e uns à direita)

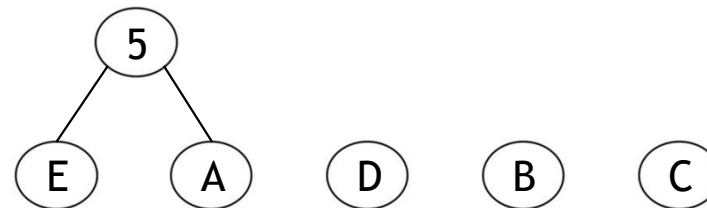
Exemplo

Texto: DCCACADEACCCCCBCEBBBD (21 bytes)

Símbolo	Frequência
A	3
B	4
C	9
D	3
E	2

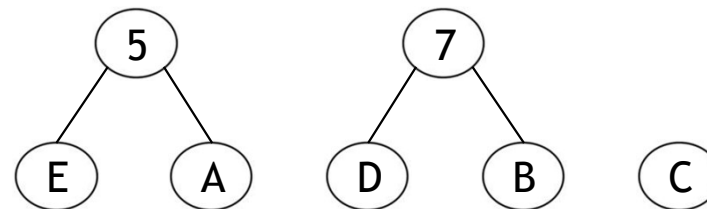
Exemplo

Símbolo	Frequência
A	3
B	4
C	9
D	3
E	2



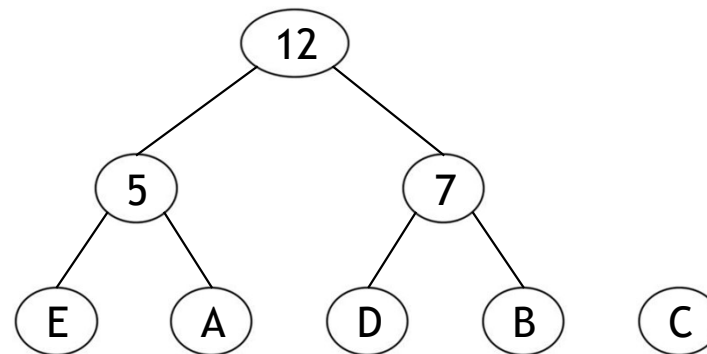
Exemplo

Símbolo	Frequência
A	3
B	4
C	9
D	3
E	2



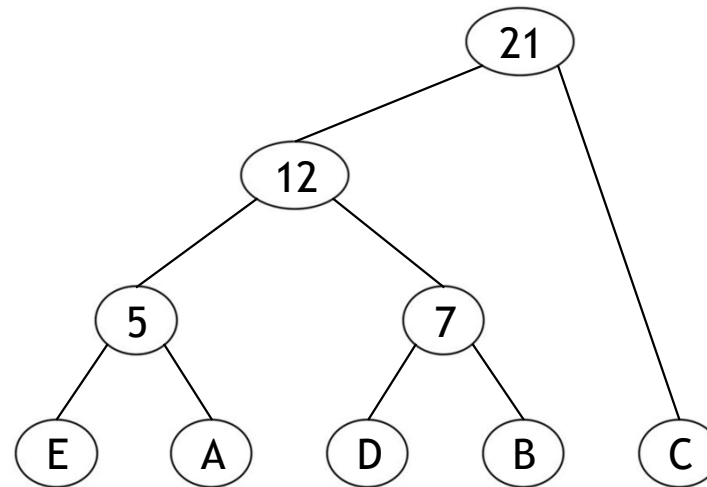
Exemplo

Símbolo	Frequência
A	3
B	4
C	9
D	3
E	2



Exemplo

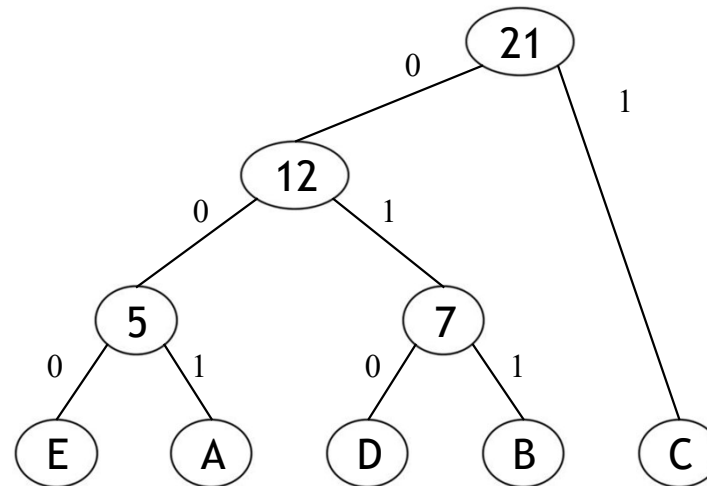
Símbolo	Frequência
A	3
B	4
C	9
D	3
E	2



Exemplo

Símbolo	Frequência	Código
A	3	001
B	4	011
C	9	1
D	3	010
E	2	000

Notem que essa árvore é um espeque da



Exemplo

Símbolo	Frequência	Código
A	3	001
B	4	011
C	9	1
D	3	010
E	2	000

Texto:

DCCACADEACCCCC
BCEBBBD (21 bytes)

Texto Codificado:

010110011001010000
001111110111000011
011011010 (7 bytes + 7
bits) ou 8 bytes

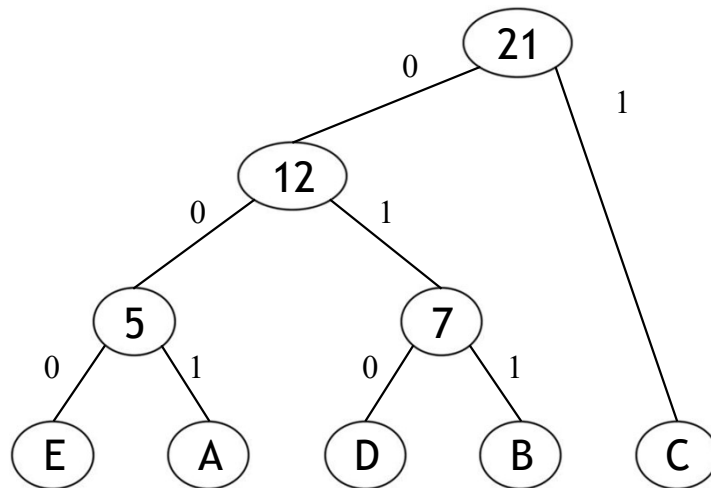


Algoritmo de Decodificação

- Uso da Árvore Binária de Prefixos
 - Percorrer o texto da esquerda para a direita, ao mesmo tempo em que a árvore é percorrida
 - Ao atingir uma folha, substituir a sequência pelo símbolo identificado, e voltar para a raiz da árvore

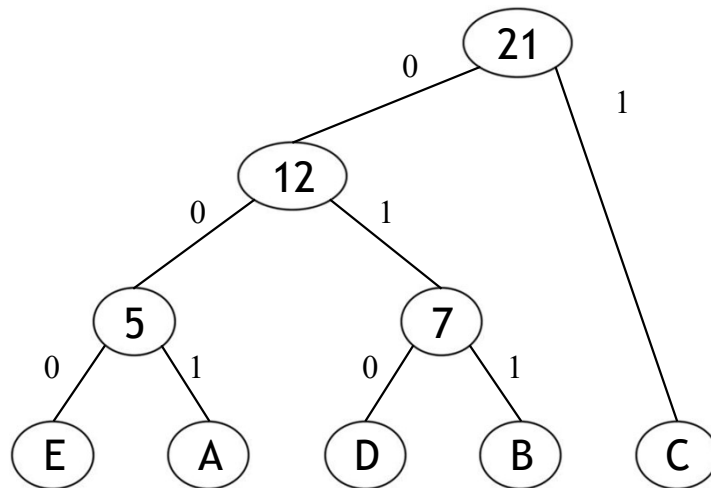
Exemplo de Decodificação

010110011001010000001111110111000011011011011010



Exemplo de Decodificação

010110011001010000001111110111000011011011011010



DCCACADEACCCCBCEBBBD

Algoritmo LZW

- Desenvolvido por Abraham Lempel e Jakob Ziv em 1977 (LZ77).
- Refinamento de LZ78 por Terry Welch em 1978.
- Huffman é um método estatística de compressão de dados.
- LZW é um método baseado em dicionários
- Diferentemente do algoritmo de Huffman, o algoritmo LZW transforma strings de tamanho variado em códigos de tamanho fixo de W bits (usualmente de 8 a 12).

Método Baseado em Dicionário

- Não usam conhecimento estatístico dos dados.
- `compress()`: a medida que a entrada é processada constrói um dicionário e utiliza os índices das strings no dicionário como código.
- `expand()`: a medida que a entrada é processada reconstrói o dicionário para reverter o trabalho de `compress()`.
- Exemplos: LZW, LZ77, Sequitur
- Aplicações: Unix compress, gzip, GIF

Algoritmo LZW compress()

input = string de entrada

crie dicionário com símbolos do alfabeto

repita

encontre o maior prefixo **s** de **input** que está no dicionário

transmita para saída o índice de **s** ponha **s+c** no dicionário onde **c** é o símbolo que segue **s** em input (*unmatched symbol*).

apague do input o prefixo **s**

até que input é vazio

LZW exemplo

Decisões de projeto:

- Alfabeto = $\{a, b\} \Rightarrow R = 2$
- $W = 3$ bits $\Rightarrow$ índices entre 0 e 7
- tamanho L do dicionário é $2^W = 8$
- índice R codificará EOF (*end of file*)

LZW compress(): exemplo

input
codificação

a	b	a	b	a	b	a	b	a	b	

dicionário

string	código
a	0
b	1
EOF	2

S=
C=a=>S
S+C=a

LZW compress(): exemplo

input
codificação

a	b	a	b	a	b	a	b	a	b	
0										

dicionário

string	código
a	0
b	1
EOF	2
ab	3

S=a
C=b=>S
S+C=ab =>dicionário

LZW compress(): exemplo

input
codificação

a	b	a	b	a	b	a	b	a	b	
0	1									

dicionário

string	código
a	0
b	1
EOF	2
ab	3
ba	4

S=b

C=a=>S

S+C=ba => dicionário

LZW compress(): exemplo

input
codificação

a	b	a	b	a	b	a	b	a	b	
0	1									

dicionário

string	código
a	0
b	1
EOF	2
ab	3
ba	4

S=a

C=b

S+C=ab=>S => dicionário

LZW compress(): exemplo

input
codificação

a	b	a	b	a	b	a	b	a	b	
0	1	3								

dicionário

string	código
a	0
b	1
EOF	2
ab	3
ba	4
aba	5

S=ab

C=a=>s

S+C=aba => dicionário

LZW compress(): exemplo

input
codificação

a	b	a	b	a	b	a	b	a	b	
0	1	3								

dicionário

string	código
a	0
b	1
EOF	2
ab	3
ba	4
aba	5

S=a
C=b
S+C=ab=>s

LZW compress(): exemplo

input
codificação

a	b	a	b	a	b	a	b	a	b	
0	1	3								

dicionário

string	código
a	0
b	1
EOF	2
ab	3
ba	4
aba	5

S=ab
C=a
S+C=aba=>S

LZW compress(): exemplo

input
codificação

a	b	a	b	a	b	a	b	a	b	
0	1	3		5						

dicionário

string	código
a	0
b	1
EOF	2
ab	3
ba	4
aba	5
abab	6

S=aba

C=b=>S

S+C=abab =>dicionário

LZW compress(): exemplo

input
codificação

a	b	a	b	a	b	a	b	a	b	
0	1	3		5			4		1	

dicionário

string	código
a	0
b	1
EOF	2
ab	3
ba	4
aba	5
abab	6
bab	7

S=b
C=a
S+C=ba=>S

LZW compress(): exemplo

input
codificação

a	b	a	b	a	b	a	b	a	b	
0	1	3		5			4		1	

dicionário

string	código
a	0
b	1
EOF	2
ab	3
ba	4
aba	5
abab	6
bab	7

S=ba

C=b=>S

S+C=bab +dicionário

LZW compress(): exemplo

input
codificação

a	b	a	b	a	b	a	b	a	b	
0	1	3		5			4		1	2

dicionário

string	código
a	0
b	1
EOF	2
ab	3
ba	4
aba	5
abab	6
bab	7

S=B

C=EOF

S+C=BEOF fim do arquivo não
precisa adicionar no arquivo

LZW compress(): exemplo

Resumo

Fomos de $8 \times 10 = 80$ bits para representar

a b a b a b a b a b

para $W \times 7 = 3 \times 7 = 21$ bits:

000 001 011 101 100 001 010

Taxa de compressão = $21/80 = 0.2625 \sim$
26%

Algoritmo LZW Expand()

crie dicionário com símbolos do alfabeto decodifique o primeiro índice
code para a string val coloque **val+?** no dicionário **repita**
 decodifique o primeiro símbolo do próximo índice code
 complete com **c** a última entrada da tabela
 termine de decodificar code e obtenha a string **s**
 transmita **s** para a saída ponha **s+?** no dicionário
• **até que** input é vazio

LZW expand(): exemplo

msg codificada
output

0	1	3	5	4	1	2

dicionário

código	string
0	a
1	b
2	EOF

sW =
cW = 0 => sW

String.SW=
String.cW=a=>output

S=
C=a
S+C=a?

LZW expand(): exemplo

msg codificada
output

0	1	3	5	4	1	2
a						

dicionário

código	string
0	a
1	b
2	EOF
3	a?

sW =0
cW =1=>sW

String.SW=a
String.cW=b=>output

S=a
C=b
S+C=ab => dicionário

LZW expand(): exemplo

msg codificada
output

0	1	3	5	4	1	2
a	b					

dicionário

código	string
0	a
1	b
2	EOF
3	ab

sW = 1
cW = 3

String.SW=**b**
String.cW=ab=>output

S=b
C=a
S+C=ba=>dicionario

LZW expand(): exemplo

msg codificada
output

0	1	3	5	4	1	2
a	b					

dicionário

código	string
0	a
1	b
2	EOF
3	ab
4	b?

sW = 1
cW = 3 = sW

String.SW = **b**
String.cW = **a**b => output

S = b
C = a
S + C = ba + dicionario

LZW expand(): exemplo

msg código
output

0	1	3		5	4	1	2
a	b	a	b				

dicionário

código	string
0	a
1	b
2	EOF
3	ab
4	ba
5	ab?

sW =3

cW = 5=>SW

String.SW=**ab**

String.cW=**a**b?

S=ab

C=a

S+C=aba=>dicionário=>output

LZW expand(): exemplo

msg código
output

0	1	3		5			4	1	2
a	b	a	b	a	b	a			

dicionário

código	string
0	a
1	b
2	EOF
3	ab
4	ba
5	aba

sW =5
cW =4

S=aba
C=b
S+C

String.SW=**aba**
String.cW=**ba**

LZW expand(): exemplo

msg código
output

0	1	3		5			4	1	2
a	b	a	b	a	b	a			

dicionário

código	string
0	a
1	b
2	EOF
3	ab
4	ba
5	aba
6	aba?

sW =5

cW =4 =>sW

String.SW=**aba**

String.cW=**ba**

S=aba

C=b

S+C=abab=>dicionário

LZW expand(): exemplo

msg código
output

0	1	3		5			4		1	2
a	b	a	b	a	b	a	b	a		

dicionário

código	string
0	a
1	b
2	EOF
3	ab
4	ba
5	aba
6	aba?

sW =5

cW =4 =>sW

String.SW=aba

String.cW=ba =>output

S=aba

C=b

S+C=abab=>dicionário

LZW expand(): exemplo

msg código
output

0	1	3		5			4		1	2
a	b	a	b	a	b	a	b	a		

dicionário
código string

0	a
1	b
2	EOF
3	ab
4	ba
5	aba
6	abab
7	ba?

sW =4
cW =1

String.SW=**ba**
String.cW=**b**

S=ba
C=b
S+C=bab=>dicionário

LZW expand(): exemplo

msg código
output

0	1	3		5			4		1	2
a	b	a	b	a	b	a	b	a	b	

dicionário

código	string
0	a
1	b
2	EOF
3	ab
4	ba
5	aba
6	abab
7	ba?

sW =4
cW =1

String.SW=**ba**
String.cW=**b** => **output**

S=ba
C=b
S+C=bab =>dicionario

LZW expand(): exemplo

msg código
output

0	1	3		5			4		1	2
a	b	a	b	a	b	a	b	a	b	

dicionário

código	string
0	a
1	b
2	EOF
3	ab
4	ba
5	aba
6	abab
7	bab

Exercícios

- 1) Escrever o texto compactado correspondente ao texto abaixo usando:
 - a) Algoritmo de Frequência de Caracteres (omitindo o número 1 para ocorrências de um único caractere)
 - b) Algoritmo de Huffman

BBBCCCCDEFFFBAAAAAAAAAAACDDFFFFFFFFCF

Exercícios

- 1) Escrever o texto compactado correspondente ao texto abaixo usando o LZW:

c a b b a c a b b

Referência

Szwarcfiter, J.; Markezon, L. Estruturas de Dados e seus Algoritmos, 3a. ed. LTC. Seção 12.5 em diante.